

Modern Code - Variables, Types and Expressions

ModernCode | MTD.ba VZ SS25

Hagenberg 13.10.2025

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Variables, Types, and Expressions

- **Why variables matter:** everything builds on data and expressions
- Variables are the **foundation** of all programming - they store the information your program works with
- Without variables: no game scores, no player health, no calculations, no meaningful programs
- Variables give **meaning** to numbers and data - **playerHealth** is clearer than just **100**
- They enable **change** - values can be updated as your program runs
- **Data types matter:** choosing **int** vs **double** affects precision, memory, and prevents errors

Variables: Names, Meaning, and Purpose

- **Variable = named storage** for a value (with a type)
- Three key operations: declaration, initialization, modification
- **Naming conventions** make code readable and professional
- **Good names** make code self-documenting

Variable Declaration

- **Declaration:** creates storage space for a value
- Syntax: `type variableName;`
- Variable exists but has no meaningful value yet

```
1      int playerHealth;      // declares storage for health
2      double damage;         // declares storage for damage
3      boolean isAlive;       // declares storage for status
```

Note: Uninitialized variables contain default values for their type.

Variable Initialization

- **Initialization:** gives a variable its first meaningful value
- Can be done with declaration or separately
- Always initialize variables before using them

```
1      // Declaration + initialization in one line
2      int playerHealth = 100;
3      double damage = 15.5;
4
5      // Declaration first, then initialization
6      int score;
7      score = 0;  // now it's safe to use
```

Variable Modification

- **Modification:** changes the value of an existing variable
- Variable must be declared and initialized first
- New value must match the variable's type

```
1      int playerHealth = 100;
2      playerHealth = 85;      // player took damage
3      playerHealth = 95;      // player healed a bit
4
5      double damage = 15.5;
6      damage = damage * 1.2; // 20% damage boost
```

Naming Conventions

- **lowerCamelCase:** first word lowercase, subsequent words capitalized
 - Different programming languages have different naming conventions
- **Meaningful names:** describe what the variable stores
- **Professional code:** follows industry standards

Good Names	Poor Names
playerHealth	ph
damagePerSecond	dps
isGameActive	flag
frameCount	fc

Example 1: Boolean (Simple)

```
1    boolean debugMode = false;    // feature flag
2    debugMode = true;             // enable debugging
```

Exercise: What is the final value of `debugMode`?

Example 1: Boolean (Simple)

```
1    boolean debugMode = false;    // feature flag
2    debugMode = true;             // enable debugging
```

Exercise: What is the final value of `debugMode`?

Solution: `debugMode = true`

Example 2: Integer (Player Health)

```
1      int playerHealth = 100;           // starting health
2      int currentHealth = playerHealth; // copy current health
3      playerHealth = playerHealth - 25;  // player takes damage
```

Exercise: What are the final values of `playerHealth` and `currentHealth`?

Example 2: Integer (Player Health)

```
1      int playerHealth = 100;           // starting health
2      int currentHealth = playerHealth; // copy current health
3      playerHealth = playerHealth - 25; // player takes damage
```

Exercise: What are the final values of `playerHealth` and `currentHealth`?

Solution:

```
-- playerHealth = 75 (100 - 25)
-- currentHealth = 100 (copied before damage)
```

Example 3: Double (Frame Rate)

```
1    double frameRate = 23.976;    // normal speed
2    frameRate = frameRate * 2;      // double speed
3    double normalSpeed = frameRate / 2; // calculate normal speed
```

Exercise: What are the final values of `frameRate` and `normalSpeed`?

Example 3: Double (Frame Rate)

```
1    double frameRate = 23.976;      // normal speed
2    frameRate = frameRate * 2;       // double speed
3    double normalSpeed = frameRate / 2; // calculate normal speed
```

Exercise: What are the final values of `frameRate` and `normalSpeed`?

Solution:

```
-- frameRate = 47.952 (23.976 * 2)
-- normalSpeed = 23.976 (47.952 / 2)
```

What are Primitive Types?

- **Primitives** are the basic building blocks of data in Java
- They store values **directly** in memory (not references to objects)
- They are **fast** and **efficient** for simple data
- Java has 8 primitive types, each with specific purpose and size
- They cannot be nothing, e.g. they always have a value (not **null**)

Literals: Writing Values Directly

- **Literals** are the actual values you write in your code
- They represent the **raw data** that gets stored in variables
- Each primitive type has its own literal syntax
- Examples: `42`, `3.14`, `true`, `'A'`

Type	Example Literals	Notes
<code>int</code>	<code>42</code> , <code>-17</code> , <code>1000000</code>	Whole numbers
<code>double</code>	<code>3.14</code> , <code>-2.5</code> , <code>1.5e3</code>	Decimal numbers
<code>boolean</code>	<code>true</code> , <code>false</code>	Logical values

Integer Types

Type	Size	Range	Default
byte	8 bits	-128 to 127	0
short	16 bits	-32,768 to 32,767	0
int	32 bits	-2.1B to 2.1B	0
long	64 bits	-9.2Q to 9.2Q	0L

- **byte**: For very small numbers, often in file I/O and network protocols
- **short**: For small numbers, used in graphics and audio processing
- **int**: Most common integer type, good balance of range and performance
- **long**: For very large numbers (timestamps, file sizes, game scores)

Floating-Point Types

Type	Size	Range	Default
<code>float</code>	32 bits	$\pm 3.4\text{e}38$	<code>0.0f</code>
<code>double</code>	64 bits	$\pm 1.8\text{e}308$	<code>0.0</code>

- `float`: Use when you need decimal precision but want to save memory
- `double`: Default choice for decimal numbers, higher precision
- Examples: `float price = 19.99f;`, `double pi = 3.14159;`

Scientific Notation (e notation):

- `3.4e38` means 3.4×10^{38} (very large number)
- `1.8e308` means 1.8×10^{308} (extremely large number)
- Useful for representing very small or very large values
- Examples: `1.5e3 = 1500.0`, `2.5e-2 = 0.025`

Boolean Type

Type	Size	Range	Default
<code>boolean</code>	1 bit	<code>true/false</code>	<code>false</code>

- **boolean**: Logical values, used in conditions and flags
- Only two possible values: **true** or **false**
- Perfect for game states: **isGameActive**, **playerAlive**
- Used in control structures (coming in next session)
- Cannot be converted to numbers (unlike some other languages)

Character Type

Type	Size	Range	Default
<code>char</code>	16 bits	0 to 65,535	<code>\u0000</code>

- **char**: Single Unicode characters
- Is also a **numeric type** (16 bits)
- Represents one character at a time (not a string)
- Examples: `'A'`, `'5'`, `'@'`, `' '`
- Used for single key presses, menu selections, symbols
- Note: We'll cover strings in detail later in the course

Special Case: void

- **void** is a special keyword, not a primitive type
- Used to indicate that a method returns **nothing**
- Cannot be used as a variable type
- Examples: `public void main()`, `private void updateScore()`

```
1      public void printMessage() {
2          IO.println("Hello World!");
3          // no return statement needed
4      }
5
6      // void cannot be used for variables:
7      // void result; // This would cause a compilation error
```

Special Case: null

- **null** is a special literal that means "no object"
- Can only be assigned to **reference types** (objects, arrays)
- **Cannot** be assigned to primitive types (**int**, **double**, **boolean**)
- Default value for all reference types

```
1      // null can be used with reference types:
2      String playerName = null;           // no name assigned yet
3      int[] scores = null;                // no array created yet
4
5      // null CANNOT be used with primitives:
6      // int health = null;                // compilation error
7      // Primitives have default values instead:
8      int health = 0;                     // default for int
```

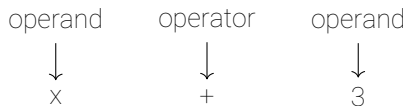
Examples by Type

```
1 // Integer literals
2 int small = 42;           // regular integer
3 int large = 2_000_000;    // underscores for readability
4 long big = 10000000000L;  // L suffix for long
5 // Floating-point literals
6 double pi = 3.14159;      // default is double
7 float speed = 25.5f;      // f suffix for float
8 double scientific = 1.5e3; // scientific notation (1500.0)
9 // Boolean and character literals
10 boolean flag = true;      // true or false
11 char grade = 'A';         // single quotes for char
```

What are Operators?

- **Operators** are special symbols that perform operations on values
- They work with **operands** (the values they operate on)
- Java has several categories of operators:
 - **Arithmetic**: + - * / % (math operations)
 - **Assignment**: = += -= *= /= (storing values)
 - **Comparison**: == != < > <= >= (comparing values)
 - **Logical**: && || ! (boolean logic)
 - **Bitwise**: & | ^ ~ « » »> (bit manipulation)

Operator and Operands



- **Operands:** The values that operators work with (x, 3)
- **Operator:** The symbol that performs the operation (+)

Operator Arity: Unary, Binary, Ternary

- **Unary operators** work on one operand:
 - $+x$ (positive), $-x$ (negative)
 - $++x$ (increment), $--x$ (decrement)
 - $!x$ (logical NOT), $\sim x$ (bitwise NOT)
- **Binary operators** work on two operands:
 - $x + y, x * y, x == y$
 - Most common type of operator
- **Ternary operator** works on three operands:
 - $x ? y : z$ (conditional operator)
 - Only one in Java (next session)

Basic Arithmetic Operators

- **Basic arithmetic**: + - * / %
- / performs division (integer vs floating-point)
- These operators work with all **numeric types**

```
1      int a = 10 + 5;      // 15
2      int b = 10 - 3;      // 7
3      int c = 4 * 6;       // 24
4      int d = 15 / 4;       // 3 (integer division)
5      int e = 15 % 4;       // 3 (remainder)
6
7      double f = 15.0 / 4; // 3.75 (floating-point division)
8      double g = 15 % 4;    // 3.0 (modulo with double result)
```

How Modulo Works

- **Modulo** (%) returns the remainder after division
- **a % b** gives the remainder when **a** is divided by **b**
- Result is always between **0** and **b-1**
- Perfect for creating cycles and wrap-around patterns

```
1  int a = 17 % 5;           // 17 % 5 = 3 remainder 2, so a = 2
2  int b = 23 % 7;           // 23 % 7 = 3 remainder 2, so b = 2
3
4  // Negative numbers (Java handles this correctly):
5  int c = -17 % 5;           // -17 % 5 = -3 remainder -2, so c = -2
```

Common Modulo Patterns I

Even/Odd Checking

Use modulo 2 to check if a number is even or odd. Even numbers have remainder 0.

```
1  boolean isEven = (score % 2 == 0);
```

Animation Cycles

Use modulo to cycle through a fixed range of values for animation frames.

```
1  int frame = tick % 4;  // cycles 0,1,2,3,0,1,2,3...
```

Common Modulo Patterns II

Wrap-around Indices

Create circular patterns like day-of-week that wraps from 6 back to 0.

```
1  int dayOfWeek = (start + offset) % 7;  // 0=Sunday, 1=Monday, etc.
```

Array Cycling

Access array elements in a circular pattern using modulo with array length.

```
1  int[] colors = {red, green, blue};  
2  int currentColor = colors[index % colors.length];
```

Increment and Decrement Operators

- ++ increments a variable by 1
- -- decrements a variable by 1
- **Pre-increment:** ++x (increment first, then use)
- **Post-increment:** x++ (use first, then increment)
- Same applies to decrement operators

```
1      int x = 5;
2      int y = ++x;      // x becomes 6, y gets 6
3      int z = x++;      // z gets 6, x becomes 7
4
5      int health = 100;
6      health--;          // health becomes 99
7      --health;          // health becomes 98
```

Assignment Operators

- = assigns a value to a variable
- += -= *= /= %= combine assignment with arithmetic
- ++ and -- are shorthand for += 1 and -= 1

```
1      int score = 100;           // basic assignment
2
3      score += 25;               // score = score + 25
4      score -= 10;              // score = score - 10
5      score *= 2;               // score = score * 2
6
7      int health = 50;
8      health++;                 // health = health + 1
9      health--;                 // health = health - 1
```

Comparison Operators

- **Equality:** `==` (equal) `!=` (not equal)
- **Relational:** `<` `>` `<=` `>=` (less, greater, less/equal, greater/equal)
- All comparison operators return **boolean** values
- Used in conditions and decision-making

```
1      int playerHealth = 75;
2      int maxHealth = 100;
3
4      boolean isFullHealth = playerHealth == maxHealth;    // false
5      boolean isLowHealth = playerHealth < 20;             // false
6      boolean canHeal = playerHealth <= maxHealth;         // true
7      boolean isDamaged = playerHealth != maxHealth;       // true
```

Logical Operators

- **&&** (AND): both operands must be **true**
- **||** (OR): at least one operand must be **true**
- **!** (NOT): inverts the boolean value
- **Short-circuit evaluation**: second operand may not be evaluated

```
1  boolean hasKey = true;
2  boolean hasWeapon = false;
3  boolean isAlive = true;
4
5  boolean canOpenDoor = hasKey && isAlive;    // true
6  boolean canFight = hasWeapon || isAlive;   // true
7  boolean isDead = !isAlive;                 // false
```

AND Operator (&&) - Truth Table

A	B	A && B
false	false	false
false	true	false
true	false	false
true	true	true

- **Logic:** Only **true** when BOTH operands are **true**
- **Game example:** Player can open door only if they have key AND are alive
- **Short-circuit:** If first operand is **false**, second is never checked

OR Operator (||) - Truth Table

A	B	A B
false	false	false
false	true	true
true	false	true
true	true	true

- **Logic:** `true` when AT LEAST ONE operand is `true`
- **Game example:** Player can fight if they have weapon OR are alive
- **Short-circuit:** If first operand is `true`, second is never checked

NOT Operator (!) - Truth Table

A	!A
false	true
true	false

- **Logic:** Inverts the boolean value
- **Game example:** `!isAlive` means player is dead
- **Double negation:** `!!isAlive` equals `isAlive`

Complex Logical Expressions

- **Combining operators:** Use parentheses to control evaluation order
- **De Morgan's Laws:** $\neg(A \ \&\& \ B)$ equals $\neg A \ || \ \neg B$
- **Game logic:** Complex conditions for game mechanics

```
1  boolean isStrong = true;
2  boolean hasWeapon = false;
3  boolean isAlive = true;
4  boolean isEnemyNearby = true;
5
6  // Can fight if (has weapon OR is strong) AND is alive
7  boolean canFight = (hasWeapon || isStrong) && isAlive;
8  // Game state check
9  boolean inDanger = isEnemyNearby && !hasWeapon && isAlive;
```

Short-Circuit Evaluation

- **Short-circuit**: Second operand may not be evaluated if result is already determined
- **&&**: If first operand is **false**, second is never checked
- **||**: If first operand is **true**, second is never checked
- **Performance benefit**: Avoids unnecessary computations

```
1  boolean hasKey = false;
2  boolean isAlive = true;
3
4  // Short-circuit: isAlive is never checked because hasKey is false
5  boolean canOpenDoor = hasKey && isAlive;      // false
```

Bitwise Operators

- `&` (AND): bit-by-bit AND operation
- `|` (OR): bit-by-bit OR operation
- `^` (XOR): bit-by-bit exclusive OR
- `~` (NOT): inverts all bits
- `<<` `>>` `>>>` (shift operators)

```
1      int flags = 0b1010;      // binary: 1010 (decimal 10)
2      int mask = 0b1100;      // binary: 1100 (decimal 12)
3      int result1 = flags & mask;  // 0b1000 (decimal 8)
4      int result2 = flags | mask;  // 0b1110 (decimal 14)
5      int result3 = flags ^ mask;  // 0b0110 (decimal 6)
6      int result4 = ~flags;        // 0b0101 (decimal -11)
7      int shifted = flags << 1;   // 0b10100 (decimal 20)
```

Type Conversion & Casting

- **Implicit widening:** `int` $\rightarrow$ `double` (automatic, safe)
- **Explicit narrowing:** `(int) 5.9` $\rightarrow$ `5` (manual, may lose data)
- **Type promotion:** Java automatically widens in expressions when needed
- **Precision matters:** choose `double` when you need fractional results

Casting Examples

```
1  // Numeric conversions
2  int health = 100;
3  double healthPercent = health; // implicit: int -> double
4  double damage = 15.7;
5  int damageInt = (int) damage; // explicit: double -> int (truncated
    to 15)
6
7  // Character conversions
8  char grade = 'A';
9  int asciiValue = grade; // implicit: char -> int (65)
10 int number = 66;
11 char letter = (char) number; // explicit: int -> char ('B')
```

Casting Placement Matters

```
1  int hits = 7;
2  int attempts = 2;
3
4  double r1 = (double) hits / attempts; // 3.5 (good: cast before
      division)
5  double r2 = (double) (hits / attempts); // 3.0 (oops: int division
      first)
6  int      r3 = (int) 5.9; // 5 (narrowing: truncates to 5)
```

Division Behavior: int vs double

- **Integer division:** When both operands are `int`, result is `int`
- **Floating-point division:** When one operand is `double`, result is `double`
- **Type promotion:** Java automatically widens `int` to `double` when needed
- **Precision loss:** Integer division truncates (cuts off) the decimal part

Arithmetic with int and double

```
1      int hits = 7;
2      int seconds = 2;
3      double time = 2.0;
4
5      int a = 3 / 2;          // 1   (int division)
6      double b = 3 / 2;      // 1.0 (int division, then widened)
7      double c = 3 / 2.0;    // 1.5 (floating division)
8      double d = 3.0 / 2;    // 1.5 (floating division)
```

Key insight: Division type depends on operand types, not result variable type.

Overflow

- **Overflow** occurs when a value exceeds the range of its data type
- Values "wrap around" to the opposite end of the range
- Common with **byte**, **short**, and **int** types

```
1  // byte range: -128 to 127
2  byte small = 127;
3  small++;                      // becomes -128 (overflow!)
4
5  // int range: -2.1B to 2.1B
6  int big = 2_147_483_647;      // maximum int value
7  big++;                       // becomes -2_147_483_648 (overflow!)
```

Fixing Overflow

- Ex. Player health becomes negative when it exceeds int maximum
- This can break game logic and cause unexpected behavior

```
1      // Game example: health overflow
2      int health = 2_147_483_647;
3      health += 1000;                // becomes negative! (bug)
4
5      // Solution: use long for very large numbers
6      long safeHealth = 2_147_483_647L;
7      safeHealth += 1000;            // works correctly
```

Bounds Checking with MAX_VALUE

- **Integer.MAX_VALUE** gives the maximum value an int can hold
- Use bounds checking to prevent overflow before it happens
- This approach keeps the original data type while preventing bugs

```
1  // Bounds checking approach
2  if (health + 1000 > Integer.MAX_VALUE) {
3      health = Integer.MAX_VALUE; // cap at maximum
4  } else {
5      health += 1000;
6  }
7
8  // Alternative: Math.min() approach
9  health = Math.min(health + 1000, Integer.MAX_VALUE);
```

Casting Overflow I

- **Casting overflow** occurs when casting a larger type to a smaller type
- Large values get truncated to fit in the smaller range
- No runtime error - values just wrap around silently

```
1  // long to int casting overflow
2  long bigNumber = 3_000_000_000L;           // 3 billion
3  int smallInt = (int) bigNumber;           // becomes -1,294,967,296
4
5  long hugeNumber = 9_000_000_000L;         // 9 billion
6  int result = (int) hugeNumber;            // becomes 410,065,408
```

Casting Overflow II

- **Preventing casting overflow** with bounds checking
- Always verify the value fits before casting
- Handle overflow cases gracefully

```
1  // Safe casting (check bounds first)
2  if (bigNumber <= Integer.MAX_VALUE && bigNumber >= Integer.
    MIN_VALUE) {
3      int safeInt = (int) bigNumber;        // safe to cast
4  } else {
5      // handle overflow case
6      IO.println("Number too large for int!");
7  }
```

Casting Best Practices

- **Use appropriate types:** Don't cast unnecessarily
- **Be aware of precision loss:** `(int) 3.9` becomes `3`
- **Watch for overflow:** `(byte) 300` becomes `44` (wraps around)
- **Game examples:**
 - Use `double` for percentages and multipliers
 - Use `int` for whole numbers (health, damage, scores)
 - Cast only when you need a specific type

Operator Precedence in Java

- **Precedence** determines the order of operations
- Higher precedence operators are evaluated first
- **Parentheses** override precedence - always evaluate first
- When in doubt, use parentheses for clarity

Java Operator Precedence Table

Precedence	Operators	Description
1 (Highest)	() [] .	Parentheses, brackets, dot
2	++ --	Postfix increment/decrement
3	++ -- + - ! ~	Prefix increment/decrement, unary
4	* / %	Multiplication, division, modulo
5	+ -	Addition, subtraction
6	<< >> >>>	Bitwise shifts
7	< <= > >=	Relational operators
8	== !=	Equality operators
9	&	Bitwise AND
10	^	Bitwise XOR
11		Bitwise OR
12	&&	Logical AND
13		Logical OR
14 (Lowest)	? : = += -=	Conditional, assignment

Precedence Examples

```
1  // Multiplication/division before addition/subtraction
2  int p1 = 2 + 3 * 4;      // 14 (3*4=12, then 2+12)
3  int p2 = (2 + 3) * 4;    // 20 (parentheses first: 5*4)
4
5  // String concatenation vs arithmetic
6  IO.println("Total: " + 2 + 3);    // "Total: 23" (left to right)
7  IO.println("Total: " + (2 + 3));  // "Total: 5" (parentheses first
    )
```

Complex Precedence: Step by Step

```
1    int p3 = 10 - 6 / 2 + 1;
```

Complex Precedence: Step by Step

```
1    int p3 = 10 - 6 / 2 + 1;
```

-- **Step 1:** Division has highest precedence

-- $6 / 2 = 3$

-- Expression becomes: $10 - 3 + 1$

Complex Precedence: Step by Step

```
1    int p3 = 10 - 6 / 2 + 1;
```

-- **Step 1:** Division has highest precedence

-- $6 / 2 = 3$

-- Expression becomes: $10 - 3 + 1$

-- **Step 2:** Subtraction (left to right)

-- $10 - 3 = 7$

-- Expression becomes: $7 + 1$

Complex Precedence: Step by Step

```
1    int p3 = 10 - 6 / 2 + 1;
```

-- **Step 1:** Division has highest precedence

-- $6 / 2 = 3$

-- Expression becomes: $10 - 3 + 1$

-- **Step 2:** Subtraction (left to right)

-- $10 - 3 = 7$

-- Expression becomes: $7 + 1$

-- **Step 3:** Final addition

-- $7 + 1 = 8$

-- $p3 = 8$

Precedence Best Practices

- **Use parentheses** for complex expressions
- **Break down** complex calculations into steps
- **Don't rely** on memorizing the entire precedence table
- **Examples:**
 - `health + armor * 0.5` VS `(health + armor) * 0.5`
 - `damage + critical * strength` VS `(damage + critical) * strength`

Expressions, Statements, and Blocks

- **Expressions:** Constructs that compute values using variables, operators, and method calls
- **Statements:** Complete units of execution (expressions terminated with semicolons)
- **Blocks:** Groups of statements between balanced braces {}
- These are the building blocks of all Java programs

Expressions

- **Expression:** Evaluates to a single value
- Can be simple or compound (combining multiple expressions)
- Data type depends on the elements used

```
1  // Simple expressions (these evaluate to values)
2  1 + 2                                // arithmetic expression
3  playerHealth > 0                     // comparison expression
4  hasWeapon && isAlive                  // logical expression
5
6  // Compound expressions
7  baseDamage * critical * strength     // multiple operations
8  (health + armor) * 0.5               // with parentheses
```

Expression Statements

- **Expression Statement:** Expression terminated with semicolon (;)
- Most common type of statement in Java
- Includes assignments, increments, and method calls

```
1  // Assignment statements
2  playerHealth = 100;           // assigns value to variable
3  score += 25;                  // compound assignment
4  // Increment/decrement statements
5  playerHealth++;               // increment by 1
6  --enemyCount;                 // decrement by 1
7  // Method invocation statements
8  IO.println("Hello World!");   // calls method
9  updateScore();                // calls custom method
```

Declaration Statements

- **Declaration Statement:** Creates variables with optional initialization
- Declares the type and name of a variable
- Can include initialization or be separate

```
1  // Variable declarations with initialization
2  int score = 0;
3  double damage = 15.5;
4  // Variable declarations without initialization
5  int playerHealth;
6  String playerName;
7  // Later initialization
8  playerHealth = 100;
9  playerName = "Hero";
```

Control Flow Statements

- **Control Flow Statement:** Controls the order of execution
- Includes conditionals, loops, and branching

```
1  // Conditional statement (if)
2  if (playerHealth <= 0) {           // condition check
3      IO.println("Game Over!");    // statement inside block
4      resetGame();                 // another statement
5  }
```

Blocks

- **Block:** Group of statements between balanced braces {}
- Can be used anywhere where a single statement is allowed
- Creates scope for variables

```
1  // Block in if statement
2  if (playerHealth < 20) {           // begin block
3      IO.println("Low health!");    // statement 1
4      int warningCount = 1;         // statement 2
5      showWarning(warningCount);    // statement 3
6  }                                 // end block
7
8  // warningCount is only visible inside the if block
9  warningCount = 2; // ERROR: warningCount not visible here
```

Why Use Constants?

- **Prevent magic numbers:** Replace hardcoded values with meaningful names
- **Maintainability:** Change value in one place, affects entire program
- **Readability:** `MAX_HEALTH` is clearer than `100`
- **Prevent bugs:** Can't accidentally change important values
- **Documentation:** Constants serve as self-documenting code

Without Constants	With Constants
<pre>if (health > 100) damage = damage * 1.5 score += 100</pre>	<pre>if (health > MAX_HEALTH) damage = damage * CRITICAL_MULTIPLIER score += POINTS_PER_KILL</pre>

Constants with `final` and `static`

```
1  static final int MAX_HEALTH = 100;
2  static final double TAX_RATE = 0.20;
3  // MAX_HEALTH = 120; // error: cannot assign a value to final
    variable
```

- **final**: Value cannot be changed after initialization
- **static**: Belongs to the class, not individual objects (more next semester)
- **Combined**: Shared constant accessible without creating objects
- Use UPPER_SNAKE_CASE for constants; improves clarity and prevents accidental changes.

int VS Integer

- **int**: primitive value; cannot be null; fast
- **Integer**: object wrapper; can be null; used with generics/collections
 - We will cover Objects and Classes next semester in detail
- Autoboxing/unboxing convenience; beware of nulls and reference equality

Autoboxing: `int <-> Integer`

- **Autoboxing**: Java automatically converts between primitives and wrappers
- **Boxing**: `int -> Integer` (automatic)
- **Unboxing**: `Integer -> int` (automatic)

```
1  // Autoboxing examples
2  int primitive = 42;
3  Integer wrapper = primitive;    // int -> Integer (boxing)
4  int backToPrimitive = wrapper;  // Integer -> int (unboxing)
5  // Mixed operations
6  Integer score = 100;            // Integer object
7  int bonus = 25;                // int primitive
8  Integer total = score + bonus;  // Integer + int = Integer (boxing)
9  int total2 = score + bonus;     // Integer + int = int (unboxing)
```

Short-Circuit Evaluation with Integer

- **Problem:** Integer can be **null**, causing runtime errors
- **Solution:** Use short-circuit evaluation to check for null first

```
Integer playerHealth = null; // Integer can be null
Integer maxHealth = 100;
```

Danger of NullPointerException:

```
if (playerHealth > 0 && playerHealth < maxHealth) {
    IO.println("Health is valid");
}
```

Using relational operators on a **null** reference causes a runtime crash.

Safe Null Check:

```
if (playerHealth != null && playerHealth > 0 && playerHealth < maxHealth) {
    IO.println("Health is valid");
}
```

Integer Nulls, and Equality

```
1  Integer x = null;
2  // int y = x; // NullPointerException at runtime (unboxing null)
3
4  Integer a = 128, b = 128;
5  IO.println(a == b);           // false (different objects)
6  IO.println(a.equals(b));      // true
7
8  Integer c = 42, d = 42;
9  IO.println(c == d);           // true (small cache) -> don't rely on
    this
```

Common Errors & Quick Fixes I

Uninitialized Variables

Reading a variable before it has been assigned a value causes compilation errors.

```
1  int playerHealth;  
2  IO.println(playerHealth); // Error: variable not initialized  
3  
4  // Fix: Initialize before use  
5  int playerHealth = 100;  
6  IO.println(playerHealth); // Works: prints 100
```

Common Errors & Quick Fixes II

Type Mismatch

Assigning a larger type to a smaller type without casting causes compilation errors.

```
1  double damage = 15.7;
2  int damageInt = damage;           // Error: possible loss of
    precision
3
4  // Fix: Explicit casting
5  int damageInt = (int) damage;     // Works: truncates to 15
6  // Or use appropriate type
7  double damageDouble = damage;    // Works: no precision loss
```

Common Errors & Quick Fixes III

Integer Division Surprises

Division between integers results in integer division, truncating decimal parts.

```
1  int hits = 7;
2  int attempts = 2;
3  int accuracy = hits / attempts;    // Result: 3 (not 3.5)
4
5  // Fix: Use double or cast before division
6  double accuracy = (double) hits / attempts;  // Result: 3.5
7  double accuracy2 = hits / 2.0;              // Result: 3.5
```

Common Errors & Quick Fixes IV

String vs Numeric Addition

The + operator behaves differently with strings vs numbers, leading to unexpected results.

```
1  int score = 100;
2  int bonus = 25;
3  IO.println("Score: " + score + bonus);    // "Score: 10025" (
      string concatenation)
4  IO.println("Score: " + (score + bonus));  // "Score: 125" (
      addition first)
5  IO.println(score + bonus + " is total");  // "125 is total" (left
      to right)
```

Summary: Variables, Types, and Expressions

Variables & Operations

- Declaration, initialization, modification
- Naming conventions (lowerCamelCase)
- Constants with **final**

Primitive Types

- **int**, **double**, **boolean**, **char**
- **byte**, **short**, **long**, **float**
- Literals and default values

Operators

- Arithmetic: **+** **-** ***** **/** **%** **++** **-**
- Assignment: **=** **+=** **-=** ***=** **/=**
- Comparison: **==** **!=** **<** **>** **<=** **>=**
- Logical: **&&** **||** **!**
- Bitwise: **&** **|** **^** **~** **<<** **>>**

Key Concepts

- Operator precedence & parentheses
- Type conversion & casting
- Overflow prevention
- Short-circuit evaluation